

Supplementary Material for: When Does Population-Mobility OD Injection Help Traffic Forecasting? A Capacity-Gap Account and a Validation-Gated Remedy

Sumin Han

This document contains Appendices A–E referenced from the main paper. Table and figure numbering continues from the main paper (Table 9 / Figure 7 are the main paper’s last table and figure), so table/figure references below match those used in the main text.

APPENDIX A SIGNAL CONTROLS PER FOLD AND FUSION- ARCHITECTURE ABLATION

Figure 8 shows the signal controls of the main paper’s Table 5 fold by fold.

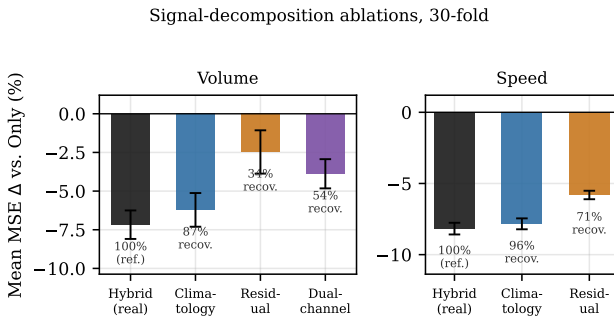


Fig. 8. Signal-decomposition conditions of the main paper’s Table 5, 30-fold mean MSE change vs. the no-injection baseline (error bars: 1 SEM), with each condition’s recovery relative to the real signal (Hybrid) annotated. *Left* (volume): Hybrid, climatology-only, residual-only, and the two-channel combination. *Right* (speed): Hybrid, climatology-only and residual-only (the routing-necessity and dual-channel controls have not yet been run on speed). Climatology recovers most of the gain on both tasks; residual’s recovery is more than twice as large on speed (71%) as on volume (34%).

Four architectural embellishments to the fusion mechanism were tested on the lightweight model and, following

early evidence of a consistent negative pattern, stopped after 5 folds per a pre-registered “stop early if it is getting worse” policy agreed upon before the sweep began, to avoid spending compute chasing a direction the early folds already argued against.

TABLE 10
Ablation: every tested variant ends up worse than the simple per-node/per-step gate of Section 4.3, most substantially so.

Variant	Fold	MSE vs. baseline
+gate-calendar	0	+7.00%
+encoder injection	0	+5.03%
+both above	0	+4.31%
+gate-calendar	5	−2.38%
2-channel (in+out)	0	+21.30%
2-channel (in+out)	5	+7.49%
2-channel (in+out)	10	+4.60%

Six of the seven tested points are worse than the simple baseline fusion of the main paper’s Section 4.3; the 2-channel variant, which adds a second, inflow-weighted routing channel alongside the outflow channel, is consistently the worst (up to +21% worse), suggesting that adding raw feature dimensionality to the injected signal without a correspondingly principled reason to do so mainly adds noise and optimization difficulty rather than useful information. The single exception (calendar-in-gate at fold 5, −2.38%) was not judged sufficient to overturn the pattern across the other six points, and the original, simpler design was retained for all subsequent experiments in this paper.

APPENDIX B PER-SENSOR SELECTIVE INJECTION

The main paper’s Section 7.2 summarizes this experiment; here we report it in full.

The selector. For each fold and each (architecture, task), we compute every sensor’s MSE for the plain and the injected (mechanism i) checkpoint on that fold’s *validation*

• Sumin Han is with the Korea Science Academy of KAIST, Busan, Republic of Korea (e-mail: suminhan@ksa.kaist.ac.kr).

month, and assign each sensor to whichever checkpoint was better there. This per-sensor choice is frozen and applied to the fold’s test month; no test data enters the choice and no model is retrained.

Results. Table 11 compares selective with uniform injection on all twenty combinations. Selective injection is significantly favorable in eighteen of twenty, including eight of the ten where uniform injection was significantly harmful, and never significantly harmful. Table 12 reports the resulting absolute RMSE for all eleven models. However, the main paper’s ensemble-controlled test (Table 7) shows that simply averaging the plain and injected predictions beats the selector in 21 of 22 combinations, and that for models of 95k parameters or more the gain from combining the two checkpoints is matched by combining two plain seeds. The selector is therefore best read as a way to avoid uniform injection’s harm, not as evidence that the signal carries value for large models.

Selection noise (free lunch). Choosing per sensor between *any* two checkpoints on a noisy one-month validation estimate can manufacture a gain. We measure this directly by applying the identical selector between two plain checkpoints differing only in seed and subtracting its gain (Table 13). Net of this baseline, the selective gain is significant in eleven of twenty combinations and reverses in the three that uniform injection harmed most (Graph WaveNet, MTGNN and AGCRN on volume). For our own model, the net selective gain (−5.82% volume, −7.81% speed) does not exceed uniform injection (−7.17%, −8.17%).

TABLE 11

Selective (validation-gated, per-sensor) vs. uniform node-feature injection, all twenty (architecture, task) combinations, both columns on the same extended 30-fold sweep as the main paper’s Table 6. V/S = volume/speed. Bold = significant (favorable for Selective, either direction for Uniform). STGCN and STID (first rule) were added to test whether GWNET/volume’s null result generalizes to other low-signal combinations; it does not. PDFormer, MTGNN, and AGCRN (second rule) were added to test whether selective injection recovers the three architectures most harmed by uniform injection (the main paper’s Table 6); it does, in five of six combinations.

Model (Task)	Selective Δ (wins, p)	Uniform Δ (wins, p)
DCRNN (V)	−4.06% (24/30, 0.0013)	+0.95% (14/30, 0.254)
DCRNN (S)	−1.51% (26/30, 6.2e-7)	+1.76% (11/30, 0.011)
GWNET (V)	−0.46% (16/30, 0.356)	+25.45% (1/30, 4.0e-9)
GWNET (S)	−1.87% (29/30, 1.6e-5)	+3.77% (5/30, 1.2e-5)
GTS (V)	−5.10% (27/30, 1.1e-6)	−0.19% (16/30, 0.877)
GTS (S)	−1.44% (28/30, 6.6e-10)	+3.86% (3/30, 4.8e-9)
GMAN (V)	−14.14% (27/30, 5.3e-10)	−7.53% (22/30, 0.0020)
GMAN (S)	−5.54% (30/30, 9.6e-19)	+0.19% (14/30, 0.789)
STAEformer (V)	−4.49% (23/30, 0.0009)	+0.74% (14/30, 0.484)
STAEformer (S)	−1.23% (26/30, 1.7e-6)	+4.71% (1/30, 4.6e-10)
STGCN (V)	−7.84% (27/30, 4.9e-6)	−2.09% (20/30, 0.831)
STGCN (S)	−3.73% (30/30, 4.4e-14)	+0.09% (16/30, 0.925)
STID (V)	−10.00% (30/30, 9.0e-14)	−9.02% (30/30, 1.7e-10)
STID (S)	−3.39% (30/30, 3.5e-13)	+0.24% (11/30, 0.567)
PDFormer (V)	−6.73% (29/30, 5.3e-7)	−2.70% (23/30, 0.274)
PDFormer (S)	−1.65% (29/30, 1.2e-8)	+3.25% (2/30, 1.1e-8)
MTGNN (V)	−1.37% (22/30, 0.074)	+11.07% (3/30, 5.5e-5)
MTGNN (S)	−1.71% (30/30, 2.2e-12)	+4.24% (0/30, 7.0e-14)
AGCRN (V)	−1.86% (22/30, 0.0063)	+11.92% (3/30, 4.7e-6)
AGCRN (S)	−1.45% (27/30, 8.1e-10)	+0.91% (11/30, 0.027)

B.1 Can a More Principled Threshold Do Better Than the Naive Rule?

The GWNET/volume exception (Table 11) suggests an obvious fix: require a sensor’s validation margin to exceed some noise-informed threshold before trusting it, rather than selecting on any margin however small. We calibrate each sensor’s threshold in units of that sensor’s *own* noise standard deviation – estimated leave-one-fold-out so a fold’s own noise draw never informs its own cutoff – and apply the same fixed critical value z (standard normal significance levels, not fit to this paper’s results) across every combination. The raw-MSE threshold is architecture- and sensor-adaptive by construction; the rule itself has zero free parameters tuned to this data.

Seventeen of eighteen combinations retain a significant favorable effect under the adaptive threshold, shrunk from the naive rule’s raw magnitude but not eliminated – conservatism mostly discards genuine, common signal rather than filtering noise, describing most tested combinations, not a special case. **MTGNN/volume, not GWNET/volume, is the combination where raising the threshold moves the result:** from directionally-favorable-but-null at $z=0$ (−1.37%, $p = 0.074$) to nominally significant at $z=1.645$ (−0.73%, $p = 0.028$) – the reversal an earlier version of this analysis, built on a stale cached checkpoint, had misattributed to GWNET/volume; we caught this during the extension to the additional architectures below and corrected it rather than let the error stand. GWNET/volume itself is not a reversal case at all under the corrected data: it is null at every z level tested ($p = 0.26$ – 0.40), consistent with Table 13’s finding that its raw selective result nets out as noise once the free-lunch correction is applied. This table runs $18 \times 4 = 72$ significance tests, so whether any individual result is trustworthy is a multiple-comparisons question: under Bonferroni correction ($\alpha = 6.9 \times 10^{-4}$), five combinations fail to survive at one or more z levels – DCRNN/volume, GWNET/volume (at every z), GWNET/speed (at three of four z), MTGNN/volume (at every z), and AGCRN/volume (at every z); the other thirteen survive at all four. Under the less conservative Benjamini-Hochberg FDR control ($q = 0.05$), all of these recover except GWNET/volume (which is simply null, not just Bonferroni-fragile) and MTGNN/volume at $z=1.0$ specifically ($p = 0.137$) – MTGNN/volume’s own reversal is itself an FDR-only, not Bonferroni-confirmed, result, so we report it with the same explicit caveat the original (mis-attributed) GWNET/volume finding carried: consistent across three of four thresholds rather than a single lucky draw, but not a confirmatory result. We report the Bonferroni-negative results alongside the FDR-positive ones rather than showing only the more favorable correction.

In short, the threshold refines but does not reverse the underlying picture for most combinations: for the sixteen combinations significant at every z level tested it trades effect size for statistical caution. MTGNN/volume is the one combination where the choice of threshold changes the qualitative verdict (null to nominally significant), and only under the less conservative of the two corrections; GWNET/volume and AGCRN/volume remain directionally favorable but Bonferroni-fragile at every z without

TABLE 12

Absolute performance before and after selective injection, 30-fold mean, all eleven models ordered by parameter count. RMSE (inj.) uses the validation-gated selector of Appendix B for every model, including our own. MAPE and r are for the plain model. Bold marks the best value per column.

Model	Params	RMSE (plain)	Volume RMSE (inj.)	MAPE	r	RMSE (plain)	Speed RMSE (inj.)	MAPE	r
STID	31–35k	360.7	342.5	15.68%	0.988	3.30	3.25	9.61%	0.969
GMAN	55–59k	504.8	468.5	24.99%	0.976	3.46	3.36	10.18%	0.966
Ours (lightweight)	62k	410.6	390.7	17.95%	0.984	4.03	3.84	12.50%	0.953
STGCN	62–67k	382.5	367.6	16.52%	0.986	3.33	3.26	9.61%	0.968
PDFormer	95–157k	367.8	355.6	14.70%	0.987	3.20	3.17	9.23%	0.971
STAEformer	160–217k	357.1	349.2	13.96%	0.988	3.18	3.16	9.07%	0.971
MTGNN	335–349k	350.1	347.6	14.64%	0.988	3.19	3.16	9.20%	0.971
DCRNN	382–386k	355.8	348.9	15.17%	0.988	3.21	3.18	9.27%	0.971
GTS	395–422k	357.0	347.9	15.38%	0.988	3.21	3.19	9.31%	0.971
GWNET	673–683k	367.8	366.8	15.99%	0.987	3.21	3.18	9.20%	0.971
AGCRN	774–777k	396.3	392.4	16.86%	0.985	3.21	3.18	9.16%	0.971

TABLE 13

Free-lunch-corrected selective injection: net effect after subtracting each architecture’s own seed-noise-only selection baseline, all twenty (architecture, task) combinations in Table 11, sorted by net effect, plus our own lightweight model. p is a paired t -test of the per-fold net effect against zero; bold marks Benjamini–Hochberg $q < 0.05$ across all 22 rows.

Model	Task	Raw Δ	Free lunch	Net Δ	p (net)
GMAN	volume	−14.14%	−4.16%	−9.98%	<1e-4
STID	volume	−10.00%	−1.47%	−8.52%	<1e-4
STGCN	volume	−7.84%	−2.43%	−5.40%	5.4e-04
GMAN	speed	−5.54%	−1.67%	−3.88%	<1e-4
GTS	volume	−5.10%	−1.41%	−3.69%	6.1e-04
PDFormer	volume	−6.73%	−3.04%	−3.68%	0.001
STID	speed	−3.39%	−0.67%	−2.72%	<1e-4
STGCN	speed	−3.73%	−1.11%	−2.62%	<1e-4
DCRNN	volume	−4.06%	−1.49%	−2.57%	0.021
STAEformer	volume	−4.49%	−2.25%	−2.23%	0.115
PDFormer	speed	−1.65%	−0.78%	−0.87%	0.003
MTGNN	speed	−1.71%	−0.88%	−0.84%	<1e-4
GWNET	speed	−1.87%	−1.41%	−0.47%	0.271
STAEformer	speed	−1.23%	−0.83%	−0.40%	0.141
DCRNN	speed	−1.51%	−1.21%	−0.30%	0.219
AGCRN	speed	−1.45%	−1.18%	−0.27%	0.130
GTS	speed	−1.44%	−1.24%	−0.20%	0.249
AGCRN	volume	−1.86%	−2.85%	+1.00%	0.356
MTGNN	volume	−1.37%	−2.92%	+1.55%	0.102
GWNET	volume	−0.46%	−3.68%	+3.22%	0.004
Ours	volume	−9.41%	−3.59%	−5.82%	<1e-4
Ours	speed	−9.23%	−1.42%	−7.81%	<1e-4

ever changing verdict. The methodological contribution is modest but generalizes past the original 2-combination pilot with zero free parameters tuned to this data.

That a cheap, external, per-sensor selector recovers – and, for the architectures already favorable, amplifies – the benefit in eighteen of twenty combinations indicates that the capacity-gap account developed throughout this section explains why an architecture *cannot learn this gating internally from its own training signal*, not that the underlying sensor-level heterogeneity in when injection helps is itself capacity-dependent. As a concrete illustration for STAEformer/speed (the combination with the most severe uniform-injection harm, and 396 sensors, the largest sensor set tested), we additionally verify that this per-sensor heterogeneity is real rather than an artifact of finite-sample noise: treating each sensor’s 30-fold win/loss record as a draw from a shared binomial win rate (the pooled 38.2%) produces far less

dispersion than observed – a chi-square overdispersion test rejects this null at $4.03\times$ the expected variance ($p \approx 0$) – and after Benjamini–Hochberg false-discovery-rate correction across all 396 per-sensor binomial tests, 76 sensors remain significant at $q < 0.05$: 31 significantly *helped* by injection and 45 significantly *hurt*. This split is itself evidence for, not against, a selective-gating approach: roughly a fifth of sensors have a real, individually detectable, and oppositely signed response to the same fixed injection mechanism.

Robustness check: a random-selection negative control. A natural objection is that mixing two independently trained checkpoints per sensor might improve accuracy for purely generic ensembling reasons, regardless of whether the selector’s validation-based choices are informative. We rule this out directly: for each fold and combination, we redraw 200 random per-sensor selection masks – matched to the exact number of sensors the real validation-based selector assigned to injection that fold – apply each to that fold’s test predictions, and compare the resulting distribution of mean %MSE changes to the real selector’s actual result. This control now covers all twenty combinations in Table 11. In all twenty combinations tested, the real, validation-informed result falls outside this null distribution entirely (empirical $p = 0.0000$ against 200 random draws per fold, i.e. no random mask matched or exceeded the real result in any combination). The comparison is starkest for Graph WaveNet/volume: a random per-sensor mix is, on average, +8.2% *worse* than plain (unsurprising, since uniform injection is catastrophically harmful there and a random mix still injects into most sensors indiscriminately), while the validation-informed selector reduces this to a statistically indistinguishable −0.46%. A purely generic ensembling effect would have made random and validation-informed mixing perform comparably; the large, consistent gap confirms the selector is using genuine, sensor-specific information from the validation split, not benefiting from mixing per se.

Two limits on this result: the validation-based selector chooses a different sensor subset each fold (a mean of 43–291 sensors out of 121–396 depending on combination), so this is evidence for fold-dependent heterogeneity, not a small fixed set of universally favorable sensors; and the experiment answers a narrower, more practical question than the architectural generalization study above – not whether a model’s internal mechanism learns to use OD well, but whether OD

TABLE 14

Adaptive noise-calibrated threshold vs. the naive rule ($z=0$), all eighteen (architecture, task) combinations with seed-control checkpoints available, 30 folds, volume and speed. $z=1.645$ is a conventional one-sided 95% critical value, not tuned to this data.

Model	Task	Naive ($z=0$) Δ (p)	Adaptive ($z=1.645$) Δ (p)
GMAN	volume	−14.14% (5.3e-10)	−6.03% (1.2e-7)
STID	volume	−10.00% (9.0e-14)	−4.79% (1.9e-9)
STGCN	volume	−7.84% (4.9e-6)	−3.78% (3.1e-5)
PDFormer	volume	−6.73% (5.3e-7)	−2.08% (9.3e-5)
STAEformer	volume	−4.49% (9.4e-4)	−1.78% (1.2e-4)
MTGNN	volume	−1.37% (0.074)	−0.73% (0.028)
DCRNN	volume	−4.06% (1.3e-3)	−1.23% (1.4e-3)
GWNET	volume	−0.46% (0.356)	−0.35% (0.372)
AGCRN	volume	−1.86% (6.3e-3)	−0.83% (0.018)
GMAN	speed	−5.54% (9.6e-19)	−2.31% (4.7e-12)
STID	speed	−3.39% (3.5e-13)	−1.98% (4.5e-11)
STGCN	speed	−3.73% (4.4e-14)	−2.12% (8.3e-12)
PDFormer	speed	−1.65% (1.2e-8)	−0.75% (6.0e-7)
STAEformer	speed	−1.23% (1.7e-6)	−0.70% (6.6e-6)
MTGNN	speed	−1.71% (2.2e-12)	−0.62% (2.1e-6)
DCRNN	speed	−1.51% (6.2e-7)	−0.42% (6.0e-5)
GWNET	speed	−1.87% (1.6e-5)	−0.83% (1.2e-3)
AGCRN	speed	−1.45% (8.1e-10)	−0.59% (1.3e-7)

carries real value at some sensors, detectable and exploitable with a cheap external check – which is immediately practical since it requires no architectural change, no retraining, and only one extra validation-window forward pass per already-trained checkpoint.

APPENDIX C

CROSS-DATASET REPLICATION: PROTOCOL DETAILS

Every result so far comes from one city and, for the between-architecture test, one signal (routed OD) – a skeptical reading is that the law describes reimplementing fidelity on our own private dataset, not a real property of signal injection. We test this on three independent, standard, publicly benchmarked datasets – PEMS-BAY (325 sensors, Bay Area), METR-LA (207 sensors, Los Angeles, DCRNN release), PeMSD7 (228 sensors, STGCN release, weekdays only, 44 days) – using a harder test of the law: a pure calendar climatology carrying no forecast content, injected through the same ten architectures. All three are natively 5-minute; we aggregate to hourly and match Seoul’s $P=Q=12$, since an earlier native-resolution check (effectively 1h-history/1h-horizon) found the injection effect’s *sign itself* flips on PEMS-BAY between horizons (+8.5% harmful at 1h vs. −0.2% at 12h), making horizon-matching necessary rather than optional. METR-LA is 8.1% missing at native resolution (vs. PEMS-BAY’s 0.003%); missing hours after aggregation (6.2%) are linearly interpolated per sensor rather than left as literal zeros, which we verified changes the result materially – the uninterpolated version shows a reversed, incoherent pattern the interpolated version does not.

The results are reported in the main paper’s Table 9 and discussed in its Section 7.4.

APPENDIX D

ADDITIONAL EXPLORATORY FINDINGS

Beyond the primary injection study, we report two further exploratory investigations that were not central to the

paper’s core question but that may be useful to readers pursuing related directions.

D.1 Spatial Embedding Initialization (node2vec)

Learned adaptive-graph architectures such as AGCRN and GTS (the main paper’s Section 2.1) initialize their node embeddings randomly and let the forecasting objective shape them end-to-end. We tested whether initializing these embeddings from a node2vec [1] random-walk decomposition of the sensor graph – a purely structural prior, computed without reference to any traffic label – helps. We consider three regimes: random initialization (the standard baseline), a *frozen* node2vec initialization (the embedding is fixed at its node2vec value and never updated by gradient descent), and a *fine-tuned* node2vec initialization (the embedding starts at its node2vec value but is updated normally thereafter). Across the GMAN architecture, where this comparison was carried out most thoroughly, the frozen variant consistently underperforms random initialization, while the fine-tuned variant consistently outperforms it; we generalized the same fine-tuned-initialization mechanism to DCRNN, Graph WaveNet, and GTS via a shared `_init_node_emb` helper, though we did not pursue a full rolling-origin sweep of this direction to the same depth as the paper’s central OD-injection study, and report it here as a smaller, secondary finding rather than a fully validated result. We view this result as broadly consistent with, and a useful independent data point for, the paper’s central capacity-gap theme: a structural prior computed independently of the downstream task (node2vec’s random-walk statistics, or, in the main study, the routed OD signal) is valuable as a *starting point or auxiliary signal* the model can adapt or selectively trust, but harmful when imposed rigidly (a frozen embedding, or, in Appendix A’s ablation, an over-engineered fusion mechanism) in place of what the model is able to learn on its own given the freedom to do so.

D.2 Purpose-Split OD Decomposition

Korean mobile-mobility records carry a `move_purpose` field distinguishing, among other categories, commute trips from other trip types. We built a purpose-split variant of the OD tensor that isolates the commute-purpose subset (after correcting a parsing bug in which the purpose codes were stored as float-formatted strings, e.g. "1.0", rather than the bare integers our initial filter expected, causing the commute subset to be silently empty until corrected; confirmed commute share after the fix is 15.5% of total OD volume) on the hypothesis that commute trips, being more temporally regular and route-predictable than discretionary trips, might be a more informative subset to route and inject than the full undifferentiated OD signal. We did not carry this direction through a full rolling-origin evaluation after the core research question was redirected toward the generalization study of the main paper's Section 7, and report the corrected, ready-to-use dataset and the motivating hypothesis here as a concrete direction for future work rather than as a completed result.

APPENDIX E

REIMPLEMENTATION FIDELITY CHECK ON A STANDARD BENCHMARK

The main paper's ten architectures are our own reimplementations, trained end-to-end on this paper's Seoul data; a natural concern is whether they reproduce their published accuracy at all, independent of OD injection. We check this directly on PEMS-BAY, training four of the ten (DCRNN, Graph WaveNet, STAEformer, STID) with no OD injection under this paper's own training protocol and comparing masked MAE at forecast horizons 3/6/12 (15/30/60 min) against STAEformer's published Table 1 [2], the one paper reporting all four under a single protocol.

TABLE 15

PEMS-BAY reproduction check: horizon-12 MAE gap versus published numbers (STAEformer Table 1). STID is shown both before and after correcting its architecture (see text).

Model	h3	h6	h12	Verdict
Graph WaveNet	+8.0%	+6.6%	+2.0%	reproduces
STAEformer	+9.5%	+7.6%	+7.7%	close
DCRNN	+8.8%	+6.8%	+7.8%	close
STID (orig.)	+9.3%	+11.6%	+20.4%	fails
STID (fixed)	+3.9%	+6.3%	+13.6%	close

Three of the four reproduce published accuracy within a band consistent with normal reimplementation variance (2–10%). STID did not: a +20.4% horizon-12 gap that grows sharply with horizon, unlike the other three, whose gaps are roughly flat. This pattern – fine at short horizons, degrading at long ones – is what would be expected if the model were missing periodicity information it needs to anticipate the traffic regime several hours out but can partly substitute for with short-term autocorrelation at one hour. Inspecting our STID implementation confirmed exactly this: our first version replaced STID's published discrete time-of-day/day-of-week identity embeddings – lookup tables that can represent sharp, non-sinusoidal per-bin patterns

such as weekday/weekend asymmetry, and the mechanism the model is named for – with the continuous sin/cos calendar features already used elsewhere in this pipeline's baselines, as a documented simplification that turned out not to be innocuous. Rebuilding STID with real discrete identity embeddings, together with two secondary corrections to match the published training protocol more closely (masking PEMS-BAY's zero-coded missing observations out of the training loss, not only the evaluation, and adding the learning-rate decay schedule the original training loop lacked entirely), closes most of the gap: +20.4% $\rightarrow$ +13.6% at horizon 12, and short-horizon accuracy moves in line with the other three architectures. The remaining gap is comparable in size to DCRNN's and STAEformer's and no longer grows disproportionately with horizon, which we read as "close" rather than "does not reproduce." Only the identity-embedding fix is architectural and therefore relevant beyond PEMS-BAY – the missing-observation mask is specific to PEMS-BAY's own missing-data coding (Seoul has none) and the learning-rate schedule was not carried over to the Seoul sweeps – so the correction applied to every STID result elsewhere in this paper is the identity-embedding fix alone; see the STID correction note in the main paper's Section 5.

REFERENCES

- [1] A. Grover and J. Leskovec, "node2vec: Scalable feature learning for networks," in *ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD)*, 2016.
- [2] H. Liu, Z. Dong, R. Jiang, J. Deng, J. Deng, Q. Chen, and X. Song, "Staeformer: Spatio-temporal adaptive embedding makes vanilla transformer sota for traffic forecasting," in *ACM International Conference on Information and Knowledge Management (CIKM)*, 2023.